

SORACOM Napter・AWS IoT Core・SORACOM Beamで学ぶMQTTハンズオン

Raspberry Piの遠隔保守から、証明書配置の違いを学ぶ3時間コース

項目	内容
概要	Napterで遠隔保守環境を作り、AWS IoT Core直結とSORACOM Beam経由を比較する
所要時間	約3時間（進行バッファ10分を含む）
対象	MQTT、TLS、AWS IoT Coreをこれから学ぶ方
前提	AWS側は講師が事前準備。受講者はPi、UD-LT2、SORACOMを操作

このハンズオンで学ぶこと

Raspberry PiをLTEルーター配下に接続し、SORACOM Napter経由で安全にSSH操作できる環境を作ります。その後、同じAWS IoT CoreのThing・証明書・トピックを使い、通信方式だけを切り替えます。

- Napter経由で、インターネット側から直接公開せずにRaspberry PiへSSH接続する
- AWS IoT CoreへMQTTで直接接続し、PiがX.509証明書と秘密鍵を保持する方式を体験する
- Pi上の証明書を削除し、SORACOM Beamが証明書を保持する方式へ切り替える
- PublishとSubscribeの両方向を確認し、TLS終端と運用責任の違いを説明できるようになる

重要:2つのMQTT経路はTLS終端が異なります

直結方式はPiからAWS IoT Coreまでが1本のMQTT接続です。Beam方式は、PiからBeamまでのMQTT接続と、BeamからAWS IoT CoreまでのMQTT接続に分かれます。Beam方式を「PiからAWSまで一続きのMQTT」と表現しないでください。

全体スケジュール

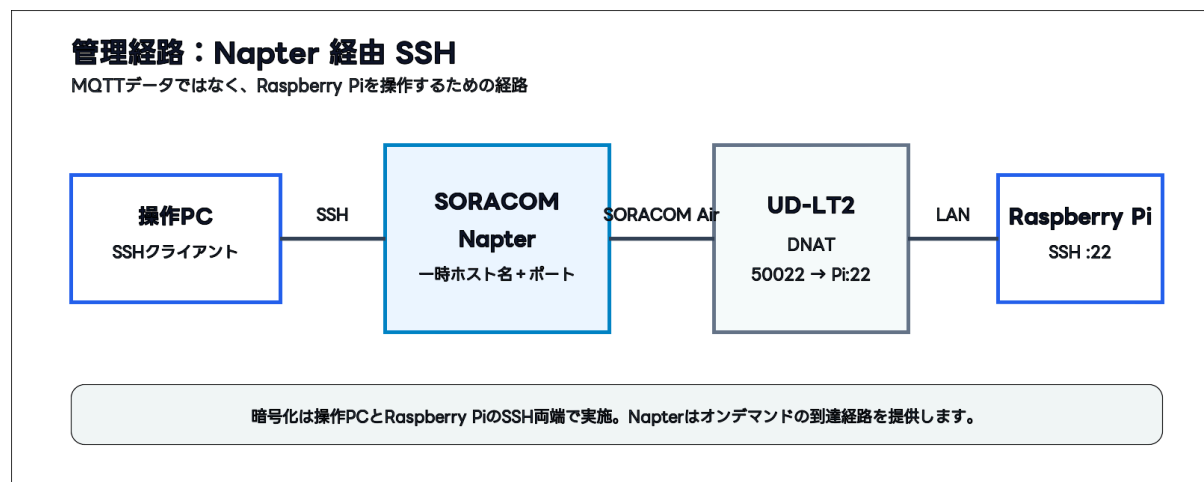
フェーズ	目安
概要・構成比較	15分

フェーズ	目安
UD-LT2、SIM、Napter経由SSH	40分
AWS IoT Core直結MQTT	40分
証明書削除・方式切替	10分
SORACOM Beam設定と双方向通信	45分
比較・トラブル対応・片付け	20分
進行バッファ	10分

0. 構成を確認する

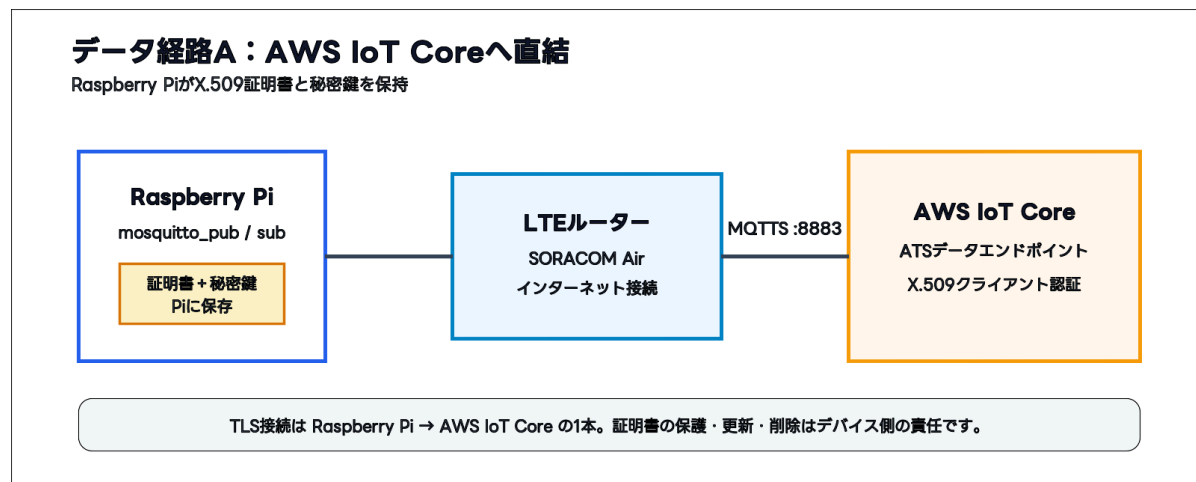
管理経路とデータ経路を分けて考えます。NapterはSSH操作のための管理経路です。MQTTメッセージはNapterを通りません。

管理経路:Napter経由SSH



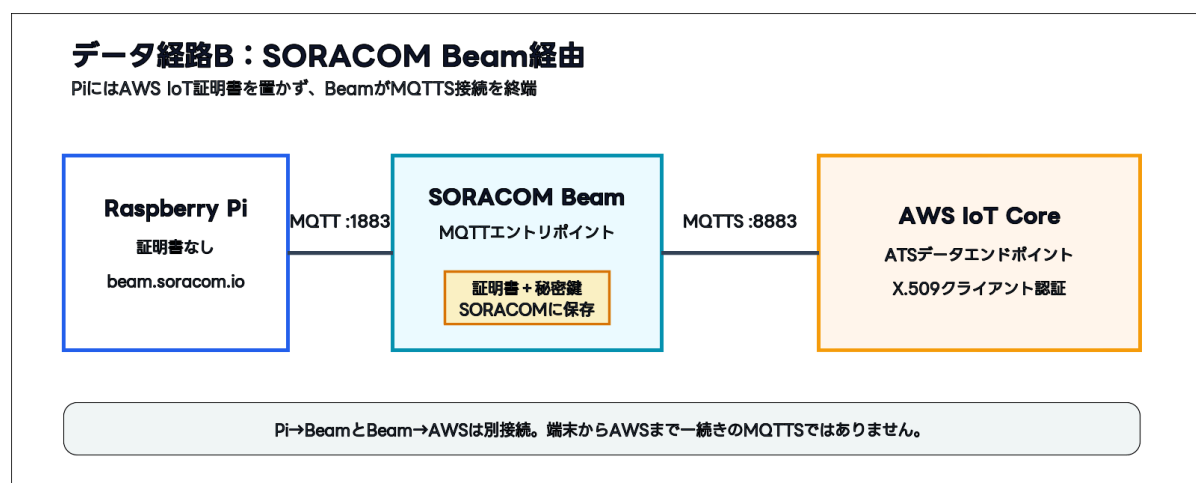
操作PCからNapterの一時的なホスト名とポートへSSH接続し、UD-LT2のDNAT設定を経由してRaspberry Piの22番ポートへ到達します。暗号化はSSHの両端で行われます。

データ経路A: AWS IoT Coreへ直結



Raspberry Piがデバイス証明書、秘密鍵、Amazon Root CA 1を保持し、AWS IoT CoreのATS エンドポイントへ8883/MQTTで接続します。

データ経路B: SORACOM Beam経由



Raspberry Piは証明書を指定せずbeam.soracom.io:1883へMQTT 3.1.1で接続します。Beam がSORACOM認証情報ストアの証明書を使い、AWS IoT Coreへ8883/MQTTで接続します。

1. ハンズオンを始める前に

Duration: 15:00

必要な機材とアカウント

- Raspberry Pi (Raspberry Pi OS Bookworm系、SSH有効)
- UD-LT2 (Web UIへログイン可能、Raspberry Piと有線LAN接続)
- SORACOM SIM 1枚 (キットごとに分離)
- OpenSSHクライアントを利用できる操作PC

- SORACOMユーザーコンソールを操作できるアカウント
- 講師から配布されたキット別AWS IoT証明書パッケージ

キット別の識別子

項目	キット別の値
Thing / MQTT Client ID	mqtt-handson-kitNN
SIMグループ	mqtt-handson-kitNN
SORACOM認証情報ID	awsiot-kitNN
Publishトピック	mqtt-handson/kitNN/telemetry
Subscribeトピック	mqtt-handson/kitNN/command

文中のkitNNは、講師から割り当てられたkit01、kit02のような値に置き換えます。他キットの証明書、Thing名、トピックは使用しないでください。

証明書パッケージの内容

- device-certificate.pem.crt: AWS IoT Coreのデバイス証明書
- private.pem.key: 秘密鍵。チャット、共有メモ、Googleドキュメントへ貼り付けない
- AmazonRootCA1.pem: AWS IoT Coreのサーバー証明書を検証するルートCA
- kit-info.txt: Thing名、AWS IoT ATSエンドポイント、Publish/Subscribeトピック

費用と安全上の注意

本ハンズオンではSORACOM Air、Napter、Beam、AWS IoT Coreを使用します。実施時点の公式料金を確認してください。秘密鍵はキット固有とし、終了後にAWS証明書を無効化します。

2. Napter経由でRaspberry PiへSSH接続する

Duration: 40:00

2.1 SIMとLAN接続を確認する

- SORACOM SIMをUD-LT2へ正しい向きで挿入し、アンテナと電源を接続する
- Raspberry Piの有線LANをUD-LT2のLANポートへ接続する
- SORACOMユーザーコンソールのSIM管理で、対象SIMがオンラインであることを確認する
- 対象SIMの先頭チェックボックスを選び、[操作]からオンデマンドリモートアクセスを開く

2.2 NapterでUD-LT2のWeb UIへ接続する

UD-LT2のWeb UIがHTTP/80で待ち受ける前提です。オンデマンドリモートアクセスを次のように作成します。

- デバイス側ポート: 80
- アクセス可能時間: 30分
- 暗号化: IoTデバイスは平文通信を待ち受ける。クライアント接続のTLSを有効化
- アクセス元IPアドレスレンジ: 空欄。コンソールを操作しているPCのグローバルIPに自動制限

発行されたHTTPS URLを開き、講師から割り当てられたUD-LT2の管理者ユーザー名とパスワードでログインします。共通の初期パスワードは使用しません。

2.3 Raspberry PiのLAN IPを確認する

UD-LT2の[ステータス]→[LANステータス]でRaspberry Piを探し、割り当てられたIPv4アドレスをメモします。以降、この値を<PI_LAN_IP>と表記します。

2.4 UD-LT2へDNATを追加する

[転送設定]→[NAT]→[追加]を開き、以下の設定を保存します。

対象	項目	設定
UD-LT2	NAT設定	DNAT
UD-LT2	プロトコル	tcp
UD-LT2	初期アドレスタイプ	interface
UD-LT2	インターフェイス	4G
UD-LT2	初期ポート	50022
Raspberry Pi	マッピングアドレス	<PI_LAN_IP>
Raspberry Pi	マッピングポート	22

2.5 SSH用Napterエントリを作成する

- デバイス側ポート: 50022
- アクセス可能時間: 4時間
- 暗号化: IoTデバイスは暗号通信を待ち受ける(HTTPS/SSHなど)。追加TLSは無効
- アクセス元IPアドレスレンジ: 空欄。操作PCのグローバルIPに限定

表示されたNapterホスト名と公開ポートを控えます。文中では<NAPTER_HOST>、<NAPTER_PUBLIC_PORT>と表記します。

2.6 操作PCからSSH接続する

```
ssh -p <NAPTER_PUBLIC_PORT> <PI_USER>@<NAPTER_HOST>
```

初回だけホスト鍵の確認が表示されます。講師の案内と接続先を照合してyesを入力し、キットに割り当てられたRaspberry Piユーザーのパスワードでログインします。

2.7 Raspberry Piの事前チェック

```
date --iso-8601=seconds
ip -br address
getent hosts beam.soracom.io
sudo ss -lntp | grep ':22'
```

時刻が大きくずれている場合はMQTTsの証明書検証に失敗します。先へ進む前に講師へ連絡してください。

3. AWS IoT Coreへ直接MQTTs接続する

Duration: 40:00

3.1 MQTTクライアントを導入する

```
sudo apt update
sudo apt install -y mosquitto-clients
mosquitto_pub --help | head
```

3.2 証明書をPiへ転送する

操作PCで証明書パッケージのフォルダを開き、Napter経由で3ファイルを転送します。

```
scp -P <NAPTER_PUBLIC_PORT> \
  device-certificate.pem.crt private.pem.key AmazonRootCA1.pem \
  <PI_USER>@<NAPTER_HOST>:/home/<PI_USER>/
```

PiへSSH接続し、専用ディレクトリへ移動して権限を設定します。

```
mkdir -p ~/aws-iot-certs
mv ~/device-certificate.pem.crt ~/private.pem.key ~/AmazonRootCA1.pem
~/aws-iot-certs/
chmod 700 ~/aws-iot-certs
chmod 600 ~/aws-iot-certs/private.pem.key
chmod 644 ~/aws-iot-certs/device-certificate.pem.crt
~/aws-iot-certs/AmazonRootCA1.pem
ls -l ~/aws-iot-certs
```

3.3 AWS IoT Coreの到達性を確認する

```
getent hosts <AWS_IOT_ENDPOINT>
openssl s_client -connect <AWS_IOT_ENDPOINT>:8883 \
  -CAfile ~/aws-iot-certs/AmazonRootCA1.pem </dev/null 2>/dev/null | \
  grep -E 'subject=|issuer=|Verify return code'
```

Verify return code: 0 (ok)が表示されることを確認します。この確認はサーバー証明書の検証であり、デバイス証明書によるMQTT認証の確認は次のPublishで行います。

3.4 PiからPublishする

```
mosquitto_pub -d -V mqttv311 \  
-h <AWS_IOT_ENDPOINT> -p 8883 \  
--cafile ~/aws-iot-certs/AmazonRootCA1.pem \  
--cert ~/aws-iot-certs/device-certificate.pem.crt \  
--key ~/aws-iot-certs/private.pem.key \  
-i mqtt-handson-kitNN \  
-t mqtt-handson/kitNN/telemetry \  
-q 1 \  
-m '{"mode":"direct","kit":"kitNN","message":"hello"}'
```

講師はAWS IoT CoreのMQTTテストクライアントでmqtt-handson/+telemetryをSubscribeし、該当キットのmodeがdirectであることを確認します。

3.5 PiでSubscribeする

```
mosquitto_sub -d -V mqttv311 \  
-h <AWS_IOT_ENDPOINT> -p 8883 \  
--cafile ~/aws-iot-certs/AmazonRootCA1.pem \  
--cert ~/aws-iot-certs/device-certificate.pem.crt \  
--key ~/aws-iot-certs/private.pem.key \  
-i mqtt-handson-kitNN \  
-t mqtt-handson/kitNN/command \  
-q 1 -C 1
```

講師が同じキットのcommandトピックへ{"action":"ping","from":"aws-console"}をPublishします。PiにJSONが1件表示され、コマンドが終了すれば双方向確認は成功です。

4. Pi上の証明書を削除する

Duration: 10:00

4.1 証明書ファイルを削除する

次のBeam方式ではPiにAWS IoT証明書を置きません。直結テストが成功したことを講師と確認してから削除します。

```
rm -f ~/aws-iot-certs/private.pem.key \  
~/aws-iot-certs/device-certificate.pem.crt \  
~/aws-iot-certs/AmazonRootCA1.pem  
rmdir ~/aws-iot-certs
```

```
test ! -e ~/aws-iot-certs && echo 'Pi上のAWS IoT証明書を削除しました'
```

通常のファイル削除はSDカード上のフォレンジックな完全消去を保証しません。本実習ではキット固有の一時証明書を使用し、終了後にAWS側で無効化します。

4.2 証明書がないことを確認する

```
find ~ -maxdepth 2 -type f \  
  \( -name 'private.pem.key' -o -name 'device-certificate.pem.crt' -o -name \  
'AmazonRootCA1.pem' \) \  
  -print
```

何も表示されないことを確認します。操作PC側の証明書パッケージは、次のSORACOM認証情報登録に使用するため、まだ削除しません。

5. SORACOM Beam経由でAWS IoT Coreへ接続する

Duration: 45:00

5.1 SORACOM認証情報ストアへX.509証明書を登録する

- SORACOMユーザーコンソールで[セキュリティ]→[認証情報ストア]を開く
- [認証情報を登録]を選び、種別をX.509証明書にする
- 認証情報IDにawsiot-kitNNを入力する
- 秘密鍵、デバイス証明書、Amazon Root CA 1はファイル名ではなく各ファイルの内容を対応欄へ貼り付ける
- 秘密鍵の内容をチャット、共有メモ、講師用チェック表へ貼り付けない

5.2 キット専用SIMグループを作成する

グループ名をmqtt-handson-kitNNとして作成し、割り当てられたSIMだけを所属させます。別キットのSIMを同じグループへ入れないでください。

5.3 Beam MQTTエントリポイントを設定する

グループ設定の[SORACOM Beam設定]で[設定を追加]→[MQTTエントリポイント]を選びます。

項目	設定値
設定名	AWS IoT kitNN
転送先の種別	Standard
転送先プロトコル	MQTTS
転送先ホスト	<AWS_IOT_ENDPOINT>
転送先ポート	8883
クライアント証明書	有効／awsiot-kitNN

項目	設定値
プラットフォームバージョン	201912
IMSIのトピック追加	無効
トピック置換	無効

VPGを利用しているSIMで公開AWS IoTエンドポイントへ接続する場合は、VPGのInternet Gatewayが有効であることを講師が確認します。

5.4 PiからBeam経由でPublishする

Piには証明書ファイルを再配置しません。接続先とコマンドから証明書オプションが消えていることを確認します。

```
mosquitto_pub -d -V mqttv311 \
  -h beam.soracom.io -p 1883 \
  -i mqtt-handson-kitNN \
  -t mqtt-handson/kitNN/telemetry \
  -q 1 \
  -m '{"mode":"beam","kit":"kitNN","message":"hello"}'
```

講師はAWS IoT CoreのMQTTテストクライアントで受信し、同じThing・同じトピックにmodeがbeamのメッセージが届いたことを確認します。

5.5 PiでBeam経由Subscribeする

```
mosquitto_sub -d -V mqttv311 \
  -h beam.soracom.io -p 1883 \
  -i mqtt-handson-kitNN \
  -t mqtt-handson/kitNN/command \
  -q 1 -C 1
```

講師がcommandトピックへ{"action":"ping","from":"aws-console"}をPublishし、Piに1件表示されればBeam経由の双方向確認は成功です。

6. 2つの方式を比較する

Duration: 10:00

比較項目	AWS IoT Core直結	SORACOM Beam経由
Piの接続先	<AWS_IOT_ENDPOINT>:8883	beam.soracom.io:1883
Pi側プロトコル	MQTTS	MQTT
AWS側プロトコル	MQTTS	BeamからMQTTS
証明書・秘密鍵	Piに保存	SORACOM認証情報ストアに保存
TLS終端	PiとAWS IoT Core	BeamとAWS IoT Core

比較項目	AWS IoT Core直結	SORACOM Beam経由
主な運用責任	Piごとの配布・保護・更新	SORACOM側の認証情報とグループ設定

確認問題

- 秘密鍵が漏えいした場合、直結方式とBeam方式で最初に無効化・削除する場所はどこですか
- Beam方式でPiが接続するポートと、BeamがAWS IoT Coreへ接続するポートは何番ですか
- NapterのSSH管理経路とMQTTデータ経路は、どこで分離されていますか
- MQTT Client IDとThing名を一致させる理由を、IoTポリシーのiot:Connectと関連付けて説明してください

7. トラブルシューティングと片付け

Duration: 10:00

よくある症状

症状	確認ポイント	対処
Napterへ接続できない	有効期限、アクセス元IP、SIMオンライン	エントリを再作成し、操作PCのグローバルIPを確認
SSHがUD-LT2で止まる	DNATの50022、PiのLAN IP	LANステータスを再確認しDNATを修正
MQTTSの証明書エラー	Piの時刻、CA、証明書、秘密鍵	時刻同期と3ファイルの組み合わせを確認
AWSからNot authorized	証明書のACTIVE状態、Thing、IoTポリシー	関連付けとClient ID、トピックを照合
直結だけ到達しない	ATSエンドポイント、8883、DNS	エンドポイントの入力と名前解決を確認
BeamでCONNACK失敗	認証情報ID、MQTTS、8883、201912	Beam設定と証明書の対応を再確認
Beamへ接続できない	SIMグループへの所属	対象SIMをmqtt-handson-kitNNへ割り当て
VPG利用時だけBeam失敗	公開AWS IoTへのInternet Gateway	VPGのInternet Gatewayを講師が確認
Subscribeが切断される	同じClient IDの二重接続	同じキットでpub/subを同時起動せず順番に実施

受講者が行う片付け

- Napterのオンデマンドリモートアクセスを手動削除する。残しても設定時間後に自動削除される
- UD-LT2のDNATルール50022を削除する
- Pi上にAWS IoT証明書が残っていないことを再確認する
- SORACOMグループからSIMを外し、Beam MQTT設定を削除する
- awsiot-kitNN認証情報をSORACOM認証情報ストアから削除する
- 操作PCのキット別証明書パッケージを削除する

講師が行う片付け

- AWS IoT証明書をINACTIVEにし、ThingとIoTポリシーから関連付けを解除する
- 再利用しない証明書、Thing、実習用ポリシーを削除する
- AWS IoT MQTTテストクライアントのSubscribeを終了する
- 各キットの完了結果と片付け結果だけを記録し、秘密鍵やIMSIを記録しない

講師付録A: AWS事前準備

A.1 キット単位のAWS IoTリソース

準備項目	キット別の値／確認内容
Thing名	mqtt-handson-kitNN
証明書	キット固有、ACTIVE
IoTポリシー	Connect、Publish、Subscribe、ReceiveをキットのClient IDとトピックへ限定
AWS IoTエンドポイント	ap-northeast-1のATSデータエンドポイント
配布ファイル	device-certificate.pem.crt、private.pem.key、AmazonRootCA1.pem
情報ファイル	kit-info.txt
講師Subscribe	mqtt-handson/+telemetry
終了処理	証明書無効化、関連付け解除、不要リソース削除

A.2 IoTポリシー例

<AWS_ACCOUNT_ID>とkitNNを各キットの値に置き換えます。ポリシーは証明書へ、証明書はThingへ関連付けます。

```
{  
  "Version": "2012-10-17",  
  "Statement": [  
    {  
      "Effect": "Allow",  
      "Principal": "*",  
      "Action": "iot:*",  
      "Resource": "arn:aws:iot:*:  
AWS_ACCOUNT_ID:thing/kitNN/*"
```

```
{
  "Effect": "Allow",
  "Action": "iot:Connect",
  "Resource":
"arn:aws:iot:ap-northeast-1:<AWS_ACCOUNT_ID>:client/mqtt-handson-kitNN"
},
{
  "Effect": "Allow",
  "Action": "iot:Publish",
  "Resource":
"arn:aws:iot:ap-northeast-1:<AWS_ACCOUNT_ID>:topic/mqtt-handson/kitNN/telemetry"
},
{
  "Effect": "Allow",
  "Action": "iot:Subscribe",
  "Resource":
"arn:aws:iot:ap-northeast-1:<AWS_ACCOUNT_ID>:topicfilter/mqtt-handson/kitNN/com
mand"
},
{
  "Effect": "Allow",
  "Action": "iot:Receive",
  "Resource":
"arn:aws:iot:ap-northeast-1:<AWS_ACCOUNT_ID>:topic/mqtt-handson/kitNN/command"
}
]
}
```

A.3 講師の当日確認

- 開始前: 全証明書がACTIVEで、Thing・ポリシーへの関連付けが正しい
- 開始前: 各キットのkit-info.txtと証明書ファイル名が一致する
- 直結確認: mqtt-handson/+/telemetryをSubscribeし、mode=directをキットごとに確認する
- Beam確認: 同じSubscribeでmode=beamをキットごとに確認する
- 下り確認: mqtt-handson/kitNN/commandへキット別にPublishする
- 終了後: 証明書を無効化し、SORACOM側の認証情報削除を確認する

講師付録B: 実機受入テスト

本番開催前に1キットで、Napier SSH、直結Publish/Subscribe、Pi上の証明書削除、Beam Publish/Subscribe、全リソースの片付けまで通しで確認します。仕様確認だけで完了扱いにしません。

合格条件

- Napier経由でPiへSSHできる
- Piの時刻とDNSが正常で、AWS IoT ATSエンドポイントへ到達できる
- 直結PublishをAWS IoT Coreで受信できる

- 直結Subscribeで講師のメッセージを受信できる
- Pi上の証明書3ファイルが削除されている
- 証明書オプションなしのBeam PublishをAWS IoT Coreで受信できる
- Beam Subscribeで講師のメッセージを受信できる
- 原本Googleドキュメント、秘密鍵、IMSI、実AWSアカウントIDを配布資料へ混入させていない

参考資料

[SORACOM Napter: IoTデバイスにSSH接続する](#)

[SORACOM: UD-LT2に接続したデバイスへ遠隔アクセスする](#)

[SORACOM Beam: AWS IoTと接続する](#)

[SORACOM: MQTTクライアントツールを利用する](#)

[SORACOM Beam: MQTTエントリポイント](#)

[AWS IoT Core: Device communication protocols](#)

[AWS IoT Core: X.509 client certificates](#)

[AWS IoT Core: Security best practices](#)

[SORACOM: サービス料金](#)

[AWS: IoT Core料金](#)

料金は実施時点のSORACOMおよびAWS公式料金ページを確認してください。固定額は本資料に記載しません。